

AAD Mod-3

Divide & Conquer

Recurrence Relation:

$$T(n) = \begin{cases} T(1) & n=1 \\ aT(n/b) + f(n) & n>1 \end{cases}$$

2-way merge sort

→ Iterative process.

→ Merge 2 sorted lists

Algorithm Merge (A, B, m, n)

i=1, j=1, k=1

while (i ≤ m & j ≤ n)

if A[i] < B[j]

C[k++] = A[i++]

else

C[k++] = B[j++]

while (i ≤ m)

C[k++] = A[i++]

while (j ≤ n)

C[k++] = B[j++]

A & B - two lists
that are
already sorted
m & n - sizes
of A & B resp

$$T(n) = O(n \log n)$$

Strassen's Matrix multiplication

→ Matrices of dimensions $n \times n$
↳ power of 2
(Add enough zeros
if not).

$$T(n) = \begin{cases} 1 & \text{if } n < 2 \\ 7T(n/2) + cn^2 & \text{otherwise} \end{cases}$$

$$= O(n^{2.81})$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

A

B

$$P = a(f-h)$$

$$Q = h(a+b)$$

$$R = e(c+d)$$

$$S = d(g-e)$$

$$T = (a+d)(e+h)$$

$$U = (b-d)(g+h)$$

$$V = (a-c)(e+f)$$

$$C = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix}$$

$$C_1 = S + T + U - Q$$

$$C_2 = P + Q$$

$$C_3 = R + S$$

$$C_4 = P - R + T - V$$

Greedy strategy

Algorithm Greedy (a, n)

for $i = 1$ to n do

$x = \text{select}(a)$

if feasible(x) then

$\text{solution} = \text{solution} + x$



Fractional Knapsack

↑ max capacity

Algorithm Knapsack (m, n, P, W)

1) Calculate P/w for each i in n . → no. of objects

2) Sort P/w in decreasing order.

3) Initialise $res = 0$, $current = m$.

4) For $i = 1$ to n do

if $(W[i] \leq current)$

$res[i] = n[i]$

$current -= W[i]$

else

$res[i] = \text{add possible fraction of } n[i]$

break

$$T(n) = O(n)$$

Spanning tree

Max. no. of spanning trees:

→ Complete graphs: n^{n-2}

→ Others:

→ Create adjacency matrix

→ Diagonal elements = degree of nodes

→ Non diagonals = -1

→ Total no. of spanning trees = cofactor (x)

$$[\text{cofactor} = \det(A) \times \text{Tran}(A)^{-1}]$$

NOVEMBER 2022

16



Wednesday

Week 47 ■ 320-045

OCTOBER

Su	Mo	Tu	We	Th	Fr	Sa
30	31					
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29

Kruskal's algorithm

→ Consider min cost edge such that it does not form a cycle.

$$T(n) = O(|E| \log |E|)$$

Algorithm Kruskal

- 1) Sort edges in non decreasing order of edge weight.
- 2) For each edge
if (edge does not create cycle)
Add edge.

Dijkstra's algorithm

- 1) Mark source node, current dist. = 0, rest ∞ .
- 2) Set non visited node with smallest dist as current node.
- 3) When each node is selected as per step 2, add weights cumulatively.
- 4) Repeat 2 & 3 until all nodes are visited.

$$T(n) = O(V^2) \text{ or } O(E \log V)$$